

SOFTWARE TEST PLAN

Project Name: StudyFlow – Academic Life Organizer

Document Version: 1.0

Author: QA Engineer

Date: August 4, 2026

Status: Approved / Execution Phase

1. INTRODUCTION

1.1 Purpose

This Test Plan document outlines the overall strategy, scope, methodology, resources, and schedule for testing the StudyFlow application. The primary objective is to verify functional accuracy, data integrity, user interface responsiveness, application security, and overall performance before release.

1.2 System Overview

StudyFlow is a web-based academic life organization platform built on a modern full-stack architecture:

- **Frontend:** Next.js (React Framework), Tailwind CSS (Hosted on Vercel)
- **Backend:** Express.js, Node.js REST API (Hosted on Render)
- **Database:** MongoDB Cloud

2. SCOPE OF TESTING

2.1 In-Scope Features

- **Functional Testing:**
 - Authentication & Authorization (User Registration, Login, JWT Token validation, Session Persistence).
 - Course Management (Create, Read, Update, Delete operations).

- Task & Schedule Management (Task creation, completion status, deadline tracking, exam calendar).
- Dashboard Analytics & User Profile components.
- **Non-Functional Testing:**
 - **UI/UX & Responsiveness:** Cross-device viewports (Desktop 1920x1080, Tablet 768px, Mobile 375px).
 - **Security Testing:** Input Sanitization (XSS and NoSQL Injection), Authentication Guards, CORS Headers, Broken Access Control.
 - **Performance:** Google Lighthouse metrics, API response latency, and cold-start handling.
- **Test Automation:**
 - Cypress End-to-End (E2E) Test Suite covering 50+ critical scenarios.
- **Alpha & Beta Testing:**
 - Internal End-to-End execution (Alpha phase) and closed user group feedback collection (Beta phase).

2.2 Out-of-Scope Features

- Native Mobile Applications (iOS and Android Native SDKs).
- Payment Gateway Integration (Planned for future releases).

3. TESTING METHODOLOGY

Testing will follow the standard Software Testing Life Cycle (STLC) framework:

Requirement Analysis -> Test Planning -> Test Case Design -> Environment Setup -> Test Execution & Bug Tracking -> Automation -> Test Closure.

Testing Types Applied:

1. **Black-Box Testing:** Verifying user flows and business logic without inspecting source code.
2. **Gray-Box Testing:** Inspecting REST API payloads, HTTP status codes (200, 400, 401, 500), local storage, and JWT structure.
3. **Regression Testing:** Automated verification via Cypress to ensure bug fixes do not introduce new defects.

4. TEST ENVIRONMENT AND TOOLS

Category	Tool / Environment	Purpose
Hosting Environments	Vercel (Frontend), Render (Backend)	Production/Staging deployment targets
Automation Framework	Cypress (v13+)	End-to-End automation execution
API Testing	Postman / HTTP Client	Independent REST API endpoint verification
Defect Tracking	GitHub Issues	Bug tracking, severity categorization, and lifecycle management
Performance Audit	Google Lighthouse / Chrome DevTools	Web Vitals and load performance analysis
Target Browsers	Google Chrome, Mozilla Firefox, Apple Safari	Cross-browser and responsive testing

5. TEST DELIVERABLES

The following documents and artifacts will be delivered upon completion of the testing lifecycle:

1. Software Test Plan (STP)
2. Requirements Traceability Matrix (RTM)
3. Test Case Specification Suite (50+ Test Cases)
4. Defect Log / Bug Reports
5. Cypress E2E Automation Repository (GitHub)
6. Final Test Summary and Sign-Off Report

6. CRITERIA FOR TESTING

6.1 Entry Criteria

- Frontend and Backend builds are deployed and operational on Staging/Production environments.
- Database schema is migrated and REST API endpoints are responding.
- Software Test Plan and Test Case Specifications are reviewed and approved.

6.2 Exit Criteria

- 100% of defined Test Cases executed (Manual and Automated).
- Zero open defects categorized as Critical, High, or Blocker.
- Test Pass Rate meets or exceeds 95%.
- All Cypress E2E automated test scripts pass successfully in continuous run.

7. RISK AND MITIGATION STRATEGY

Risk Description	Severity	Mitigation Strategy
Backend cold starts on Render free tier causing API delays	Medium	Issue a warm-up ping request before starting automated test suites.
Viewport layout breaking on non-standard mobile screens	High	Utilize Cypress Viewport API to test breakpoints at 375px, 768px, and 1280px automatically.
SMTP server delays during integration testing	Low	Intercept and mock email service responses during E2E automated runs.